

Addressing Modes

Tanmay Bhowmik

Operand, Operator and Opcode

- Operands are the objects that are manipulated and operators are the symbols that represent specific actions.
 - For example, in the expression $x + 5$, x and 5 are operands and $+$ is an operator.
- Opcode is an instruction that tells processor what to do with the variable or data written besides it.
 - For Example: `ADD R1,R2`
 - `ADD` is the Opcode and `R1` and `R2` are the Operand.

Addressing Modes – What is...

- The term addressing modes refers to the way in which the operand of an instruction is specified.
- The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually executed.
- An addressing mode specifies how to calculate the effective memory address of an operand by using information held in registers and/or constants contained within a machine instruction

Format

- The control unit is designed to go through an instruction cycle that is divided into three major phases:
 - Fetch the instruction
 - Decode the instruction
 - Execute the instruction
- There is one register called Program Counter (PC) that keeps track of the instructions in the program stored in memory.
- An example of an instruction format with a distinct addressing mode field is shown in following figure:

Figure 8-6 Instruction format with mode field.

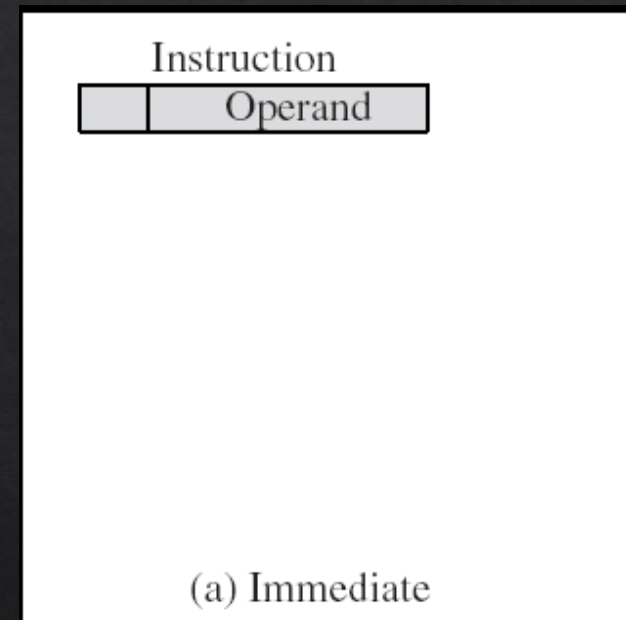


Addressing Modes – Types

- Immediate
- Direct
- Indirect
- Register
- Register Indirect
- Displacement (Indexed)
- Stack

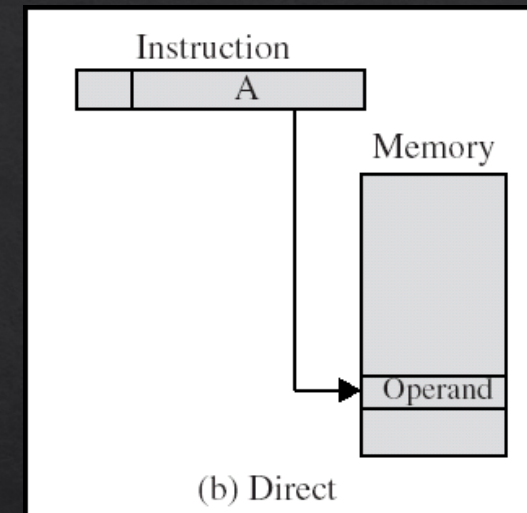
Immediate Addressing

- Operand is part of instruction
- Operand = address field
- e.g. ADD AX, 5h
- LDA #5
 - ◇ Add 5 to contents of accumulator
 - ◇ 5 is operand
- No memory reference to fetch data
- Fast
- Limited range



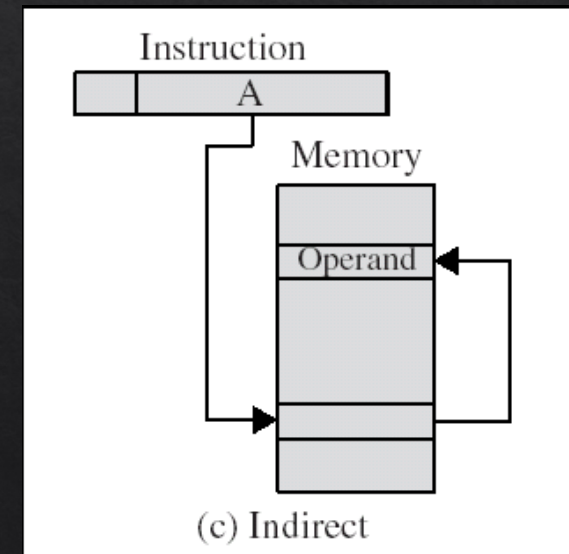
Direct Addressing

- Address field contains address of operand
- Effective address $EA = \text{address field (A)}$
- `ADD AX, value`
- Value `DB 05h`
 - ◊ Add contents of cell value to accumulator `AX`
 - ◊ Look in memory at address value for operand
- Single memory reference to access data
- No additional calculations to work out effective address
- Limited address space



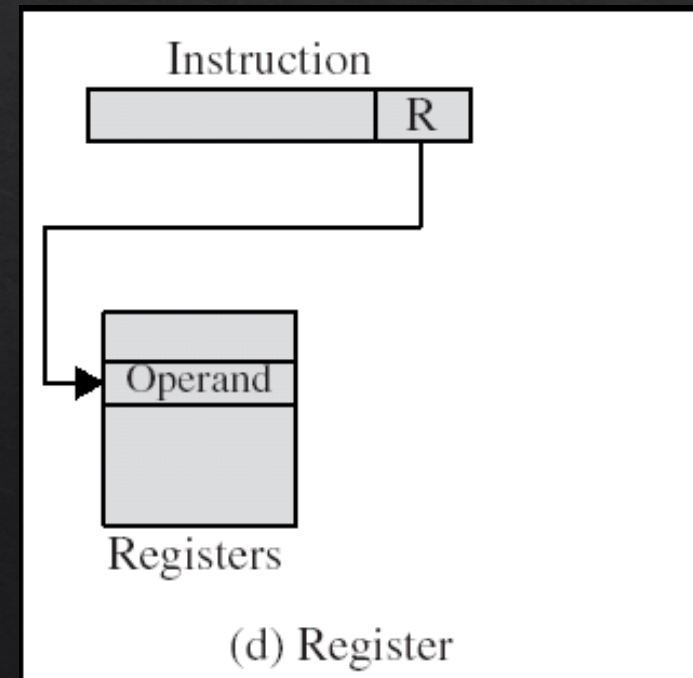
Indirect Addressing

- Memory cell pointed to by address field contains the address of (pointer to) the operand
- $EA = (A)$
 - ◊ Look in A, find address (A) and look there for operand
- e.g. `ADD AX, (A)`
 - ◊ Add contents of cell pointed to by contents of A to accumulator



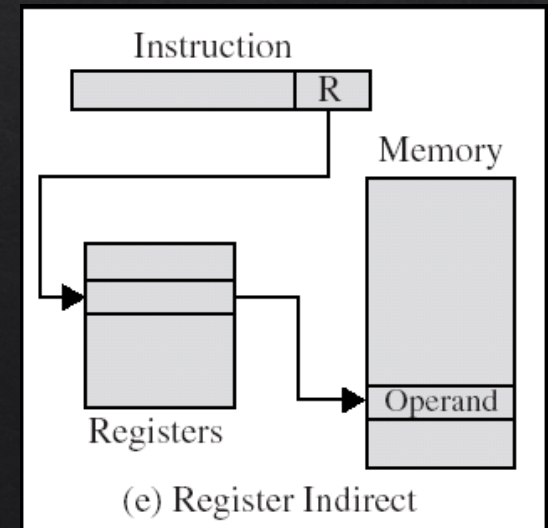
Register Addressing

- Operand is held in register named in address field
- $EA = R$
- Limited number of registers
- Very small address field needed
 - ◇ Shorter instructions
 - ◇ Faster instruction fetch
 - ◇ `MOV AX, BX`
 - ◇ `ADD AX, BX`



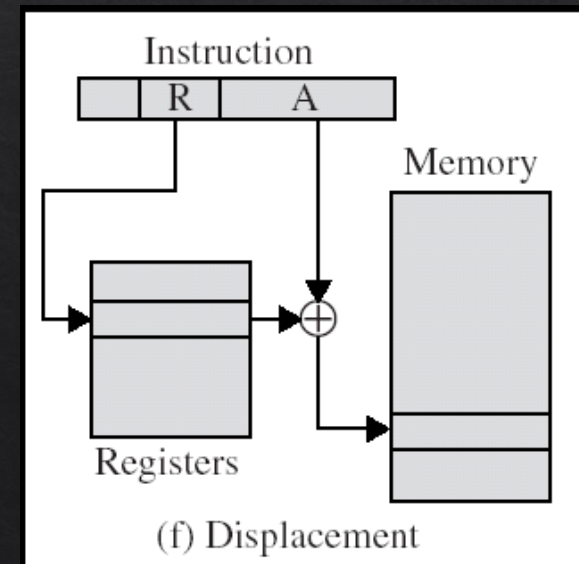
Register Indirect Addressing

- Indirect addressing
- $EA = (R)$
- Operand is in memory cell pointed to by contents of register R
- Large address space (2^n)
- One fewer memory access than indirect addressing



Displacement Addressing

- $EA = A + (R)$
- Effective address = start address + displacement
- Effective address = Offset + (Segment Register)
- Use direct and register indirect
- Address field hold two values
 - ◇ A = base value
 - ◇ R = register that holds displacement
 - ◇ or vice versa



Relative Addressing (PC-Relative)

- A version of displacement addressing
- $R = \text{Program counter, PC}$
- $EA = A + (PC)$
- i.e. get operand from A cells from current location pointed to by PC
- c.f locality of reference & cache usage

Base-Register Addressing

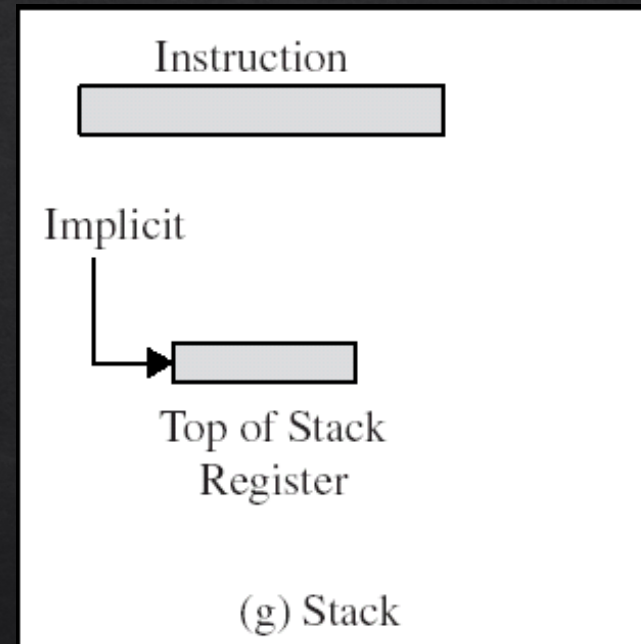
- A holds displacement
 - ◊ $EA = (CS) + A$
- R holds pointer to base address
- R may be explicit or implicit

Indexed Addressing

- $A = \text{base}$
- $R = \text{displacement}$
 - ◊ $EA = A + (R)$
- Good for accessing arrays
 - ◊ $EA = A + (R)$
 - ◊ $R++$

Stack Addressing

- Operand is (implicitly) on top of stack
- e.g.
 - ◇ ADD Pop top two items from stack and add and push



Finally...

Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory reference	Limited address space
Register indirect	EA = (R)	Large address space	Extra memory reference
Displacement	EA = A + (R)	Flexibility	Complexity
Stack	EA = top of stack	No memory reference	Limited applicability

Address		Memory	
PC = 200 R1 = 400 XR = 100 AC	200	Load to AC	Mode
	201	Address = 500	
	202	Next instruction	
	399	450	
	400	700	
	500	800	
	600	900	
	702	325	
	800	300	

Figure 8-7 Numerical example for addressing modes.

TABLE 8-4 Tabular List of Numerical Example

Addressing Mode	Effective Address	Content of AC
Direct address	500	800
Immediate operand	201	500
Indirect address	800	300
Relative address	702	325
Indexed address	600	900
Register	—	400
Register indirect	400	700
Autoincrement	400	700
Autodecrement	399	450

Relative add: PC + address
Index add: XR + address

Three-Address Instructions

Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand. The program in assembly language that evaluates $X = (A + B) * (C + D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

ADD	R1, A, B	$R1 \leftarrow M[A] + M[B]$
ADD	R2, C, D	$R2 \leftarrow M[C] + M[D]$
MUL	X, R1, R2	$M[X] \leftarrow R1 * R2$

It is assumed that the computer has two processor registers, $R1$ and $R2$. The symbol $M[A]$ denotes the operand at memory address symbolized by A .

Two-Address Instructions

Two-address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word. The program to evaluate $X = (A + B) * (C + D)$ is as follows:

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

The MOV instruction moves or transfers the operands to and from memory and processor registers. The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

One-Address Instructions

One-address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A + B) * (C + D)$ is

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

All operations are done between the AC register and a memory operand. *T* is the address of a temporary memory location required for storing the intermediate result.

Zero-Address Instructions

A stack-organized computer does not use an address field for the instructions ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack. The following program shows how $X = (A + B) * (C + D)$ will be written for a stack-organized computer. (TOS stands for top of stack.)

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C + D)$
MUL		$TOS \leftarrow (C + D) * (A + B)$
POP	X	$M[X] \leftarrow TOS$

To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse Polish notation. The name "zero-address" is given to this type of computer because of the absence of an address field in the computational instructions.